

properly authenticated session management and client-side authentications such as client certificates should be used. Another idea is to use multi-factor authentication implemented on secure protocols. As we learnt earlier, an MITM attack planted on a browser can bypass all these security measures; hence, it is important to implement the best secure coding practices to deal with such situations in the Web applications.

Linux FOSS systems come with a built-in, but usually less known feature called 'source address verification'. It is a kernel feature which, when turned on, starts dropping packets which appear to be arriving from the internal network, but in reality are not. Most of the latest kernels, such as Ubuntu and CentOS, do support it, but if your Linux distro does not, it is time to upgrade. Modifying the *hosts.conf* file to add '*nospoof on*' is another level of defence to try. In terms of detection, for smaller Linux networks, a nice utility called *arpwatch* is very useful. It keeps track of MAC and IP addresses, records all changes and can be scripted to alert administrators upon a possible attack. Scripting can also be done to go through network interface logs and look for anomalies in terms of source address forging.

(Continued from Page 84....)

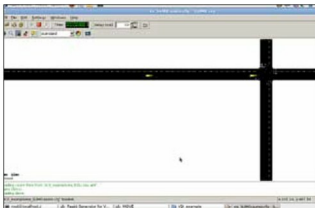


Figure 6: A close look at SUMO output

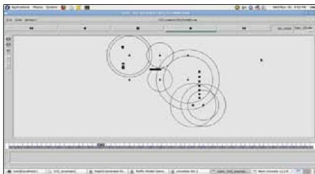


Figure 7: Sample of NS-2 animation

related aspects. Figure 4 shows the main screen of MOVE for vehicular traffic and road-map generation. Figures 5 and 6 display a sample SUMO output.

Summary

A MITM attack is really difficult to tackle, and hence should be taken seriously by the IT management team. It can result in data theft, causing severe reputation and monetary losses to firms. The bottom line is that you must have a correctly defined security perimeter defence design, ensure server and network component hardening, implement a robust patch management system, and follow the best security practices. Since this attack may not be visible, being vigilant in terms of network problems and performance always helps detect it, before a data theft can occur. **END** 

By: Prashant Phatak

The author has over 18 years of experience in the IT hardware, networking, Web technologies and IT security field. Prashant is MCSE and MCDBA certified and is also a F5 load balancer expert. In the IT security world, he is an ethical hacker and network forensic specialist. Prashant runs his own firm called Valency Networks in India (<http://www.valencynetworks.com>) providing consultancy in IT security design, security audit, infrastructure technology and business process management. He can be reached at prashant@valencynetworks.com.

Once the mobility pattern is generated, MOVE converts it into the trace files readable by NS-2 for network traffic simulation. Figure 7 shows the NS-2 animation of the mobility pattern in Figure 5.

A synopsis of the simulation process

The process involves:

- Defining the nodes and edges in the desired road topology, in the *.nod.xml* and *.edge.xml* files respectively. This can be done using the MOVE GUI, or by simply creating and editing a file.
- Next, we generate the road map, i.e., *.net.xml*, using the *net-convert* command or MOVE.
- Define the vehicle flow in the *.flow.xml* file, and generate *.rou.xml* and *sumo.cfg* files, using MOVE.
- Visualisation of the road traffic configuration can be done using *sumo-gui*.
- Now *.tr*, *.tcl* and *.nam* file can be generated, using MOVE, from the road traffic simulation.
- Necessary changes are made in the *.tcl* file for relevant network simulation or analysis.

In the next article, I plan to cover step-by-step simulation generation using SUMO and MOVE. And subsequently, I aim to share NS-2 simulation for wired and wireless networks on Fedora 14, or Ubuntu, from scratch. **END** 

By: Pooja P Shah

The author is an assistant professor at Nirma University, Gujarat and a researcher in the field of Vehicular Ad-hoc Networks. Her hobbies are teaching, learning new technologies, listening to music and writing.